Aim: To train an agent based on the environment to find the shortest path using Reinforcement Learning

IDE: Google Colab

Theory:

Reinforcement learning is an area of Machine Learning. It is about taking suitable action to maximize reward in a particular situation. It is employed by various software and machines to find the best possible behavior or path it should take in a specific situation. Reinforcement learning differs from supervised learning in a way that in supervised learning the training data has the answer key with it so the model is trained with the correct answer itself whereas in reinforcement learning, there is no answer but the reinforcement agent decides what to do to perform the given task. In the absence of a training dataset, it is bound to learn from its experience.

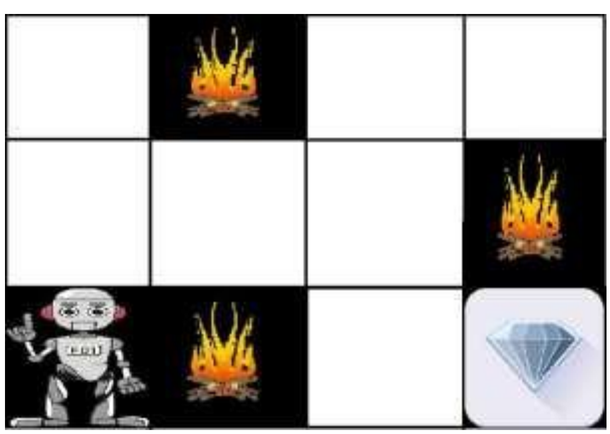
Reinforcement Learning (RL) is the science of decision making. It is about learning the optimal behavior in an environment to obtain maximum reward. In RL, the data is accumulated from machine learning systems that use a trial-and-error method. Data is not part of the input that we would find in supervised or unsupervised machine learning.

Reinforcement learning uses algorithms that learn from outcomes and decide which action to take next. After each action, the algorithm receives feedback that helps it determine whether the choice it made was correct, neutral or incorrect. It is a good technique to use for automated systems that have to make a lot of small decisions without human guidance.

Reinforcement learning is an autonomous, self-teaching system that essentially learns by trial and error. It performs actions with the aim of maximizing rewards, or in other words, it is learning by doing in order to achieve the best outcomes.

Example:

The problem is as follows: We have an agent and a reward, with many hurdles in between. The agent is supposed to find the best possible path to reach the reward. The following problem explains the problem more easily.



The above image shows the robot, diamond, and fire. The goal of the robot is to get the reward that is the diamond and avoid the hurdles that are fired. The robot learns by trying all the possible paths and then choosing the path which gives him the reward with the least hurdles. Each right step will give the robot a reward and each wrong step will subtract the reward of the robot. The total reward will be calculated when it reaches the final reward that is the diamond.

Main points in Reinforcement learning –

- Input: The input should be an initial state from which the model will start
- Output: There are many possible outputs as there are a variety of solutions to a particular problem
- Training: The training is based upon the input, The model will return a state and the user will decide to reward or punish the model based on its output.
- The model keeps continues to learn.
- The best solution is decided based on the maximum reward.

Types of Reinforcement:

There are two types of Reinforcement:

1. **Positive:** Positive Reinforcement is defined as when an event, occurs due to a particular behavior, increases the strength and the frequency of the behavior. In other words, it has a positive effect on behavior.
   Advantages of reinforcement learning are:
   - Maximizes Performance
   - Sustain Change for a long period of time
   - Too much Reinforcement can lead to an overload of states which can diminish the results
2. **Negative:** Negative Reinforcement is defined as strengthening of behavior because a negative condition is stopped or avoided.
   Advantages of reinforcement learning:
   - Increases Behavior
   - Provide defiance to a minimum standard of performance
   - It Only provides enough to meet up the minimum behavior

Reinforcement learning elements are as follows:

1. Policy
2. Reward function
3. Value function
4. Model of the environment

Policy: Policy defines the learning agent behavior for given time period. It is a mapping from perceived states of the environment to actions to be taken when in those states.
Reward function: Reward function is used to define a goal in a reinforcement learning problem. A reward function is a function that provides a numerical score based on the state of the environment
Value function: Value functions specify what is good in the long run. The value of a state is the total amount of reward an agent can expect to accumulate over the future, starting from that state.
Model of the environment: Models are used for planning.

Credit assignment problem: Reinforcement learning algorithms learn to generate an internal value for the intermediate states as to how good they are in leading to the goal. The learning decision maker is called the agent. The agent interacts with the environment that includes everything outside the agent.
The agent has sensors to decide on its state in the environment and takes action that modifies its state.

The reinforcement learning problem model is an agent continuously interacting with an environment. The agent and the environment interact in a sequence of time steps. At each time step t, the agent receives the state of the environment and a scalar numerical reward for the previous action, and then the agent then selects an action.

Reinforcement learning is a technique for solving Markov decision problems. Reinforcement learning uses a formal framework defining the interaction between a learning agent and its environment in terms of states, actions, and rewards. This framework is intended to be a simple way of representing essential features of the artificial intelligence problem.

Various Practical Applications of Reinforcement Learning –

- RL can be used in robotics for industrial automation.
- RL can be used in machine learning and data processing
- RL can be used to create training systems that provide custom instruction and materials according to the requirement of students.

Application of Reinforcement Learnings
1. Robotics: Robots with pre-programmed behavior are useful in structured environments, such as the assembly line of an automobile manufacturing plant, where the task is repetitive in nature.

2. A master chess player makes a move. The choice is informed both by planning, anticipating possible replies and counter replies.

3. An adaptive controller adjusts parameters of a petroleum refinery's operation in real time.

RL can be used in large environments in the following situations:

1. A model of the environment is known, but an analytic solution is not available;
2. Only a simulation model of the environment is given (the subject of simulation-based optimization)

3. The only way to collect information about the environment is to interact with it.

Advantages and Disadvantages of Reinforcement Learning

Advantages of Reinforcement learning

1. Reinforcement learning can be used to solve very complex problems that cannot be solved by conventional techniques.

2. The model can correct the errors that occurred during the training process.

3. In RL, training data is obtained via the direct interaction of the agent with the environment

4. Reinforcement learning can handle environments that are non-deterministic, meaning that the outcomes of actions are not always predictable. This is useful in real-world applications where the environment may change over time or is uncertain.

5. Reinforcement learning can be used to solve a wide range of problems, including those that involve decision making, control, and optimization.

6. Reinforcement learning is a flexible approach that can be combined with other machine learning techniques, such as deep learning, to improve performance.

Disadvantages of Reinforcement learning

1. Reinforcement learning is not preferable to use for solving simple problems.

2. Reinforcement learning needs a lot of data and a lot of computation

3. Reinforcement learning is highly dependent on the quality of the reward function. If the reward function is poorly designed, the agent may not learn the desired behavior.

4. Reinforcement learning can be difficult to debug and interpret. It is not always clear why the agent is behaving in a certain way, which can make it difficult to diagnose and fix problems.

Pre Lab Exercise:

1. How does reinforcement learning differ from supervised and unsupervised learning?

Reinforcement learning differs from supervised and unsupervised learning in the way it learns from data and makes decisions. In supervised learning, the model is trained using labeled data where the correct output is already known, while in unsupervised learning, the model works with unlabeled data to find patterns or groupings. In contrast, reinforcement learning learns by interacting with an environment and receiving feedback in the form of rewards or penalties. Instead of being given correct answers, it learns through trial and error to take actions that maximize long-term rewards. This makes reinforcement learning more suitable for decision-making problems, such as game playing, robotics, and autonomous systems, whereas supervised and unsupervised learning focus mainly on prediction and pattern discovery.

2. What are the key components of a reinforcement learning system and explain them.

The key components of a reinforcement learning (RL) system are:

- **Agent:**
  The learner or decision-maker that interacts with the environment and takes actions.
- **Environment:**
  The external system with which the agent interacts. It provides feedback based on the agent's actions.
- **State (S):**
  The current situation or condition of the environment at a given time.
- **Action (A):**
  The set of all possible moves or decisions the agent can take in a given state.
- **Reward (R):**
  The feedback signal received from the environment after taking an action. It indicates whether the action was good or bad.
- **Policy (π):**
  A strategy that defines how the agent chooses actions based on the current state.
- **Value Function (V):**
  It estimates how good a particular state is in terms of expected future rewards.
- **Q-Function (Q):**
  It estimates the value of taking a specific action in a given state, considering future rewards.

3. Describe the difference between exploration and exploitation in reinforcement learning.

In reinforcement learning, exploration and exploitation are two important strategies that guide how an agent learns. Exploration refers to the process of trying new or less familiar actions to gain more knowledge about the environment and discover potentially better rewards. In contrast, exploitation involves using the agent's existing knowledge to choose the action that currently provides the highest reward. The main difference is that exploration focuses on learning and discovering new possibilities, while exploitation focuses on maximizing performance based on what is already known. A balanced approach between the two is essential, as too much exploration can lead to inefficiency, while too much exploitation may cause the agent to miss out on better opportunities.

4. What is the purpose of Q-Learning in reinforcement learning?

The purpose of Q-learning in reinforcement learning is to enable an agent to learn the optimal actions to take in different situations in order to maximize cumulative rewards over time. It does this by learning a Q-value (quality value) for each state-action pair, which represents how good it is to take a particular action in a given state. By continuously updating these Q-values based on the rewards received and future expected rewards, the agent gradually improves its decision-making without needing a model of the environment. Ultimately, Q-learning helps the agent develop an optimal policy, allowing it to choose the best possible action in each state to achieve long-term success.

Program (Code):

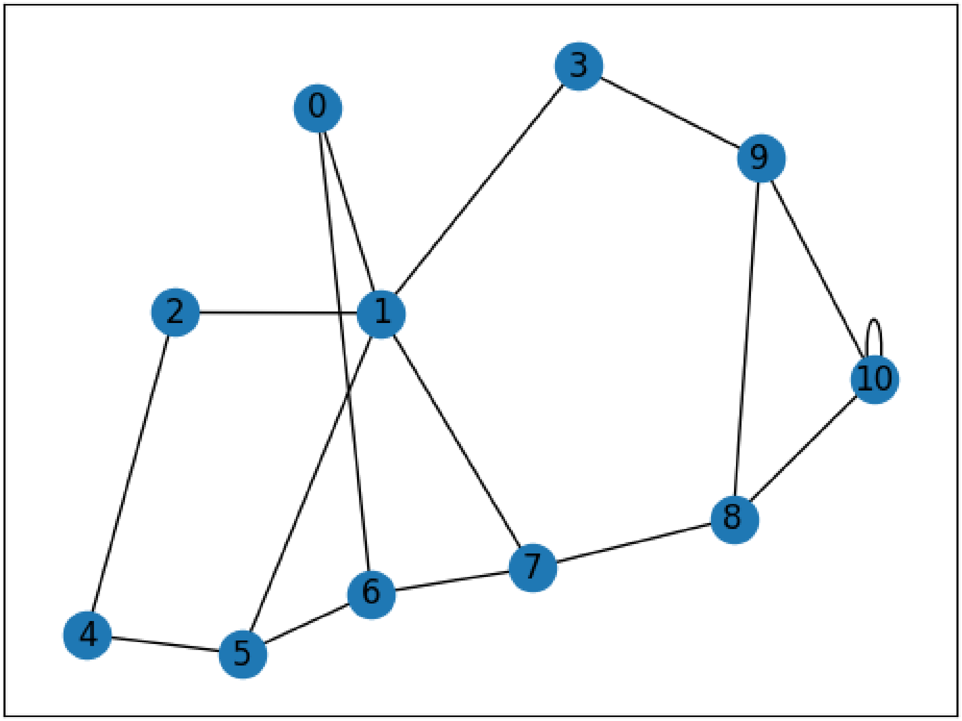
To be attached with

```
import numpy as np
import pylab as pl
import networkx as nx
edges = [(0, 1), (1, 5), (5, 6), (5, 4), (1, 2),
    (1, 3), (9, 10), (2, 4), (0, 6), (6, 7),
    (8, 9), (7, 8), (1, 7), (3, 9),(10,8),(10,10)]

goal = 10
G = nx.Graph()
G.add_edges_from(edges)
pos = nx.spring_layout(G)
nx.draw_networkx_nodes(G, pos)
nx.draw_networkx_edges(G, pos)
nx.draw_networkx_labels(G, pos)
pl.show()
MATRIX_SIZE = 11
R = np.matrix(np.ones(shape =(MATRIX_SIZE, MATRIX_SIZE)))
R *= -1

for point in edges:
  print(point)
  if point[1] == goal:  #GOAL IS 10
    R[point] = 100
  else:
    R[point] = 0

  if point[0] == goal:
    R[point[::-1]] = 100    #SEQUENCE [START: STOP: STEP] shows the output in reverse order.  (9->10) gets reward
but (10,9) doesn't
  else:
     R[point[::-1]]= 0   #REVERSE ALSO SAME POINT AS O
    # reverse of point

R[goal, goal]= 100
print(R)
# add goal point round trip
 Q = np.matrix(np.zeros([MATRIX_SIZE, MATRIX_SIZE]))
```

```
gamma = 0.75
# learning parameter
initial_state = 1

# Determines the available actions for a given state
def available_actions(state):
  current_state_row = R[state, ]
  available_action = np.where(current_state_row >= 0)[1]
  return available_action

available_action = available_actions(initial_state)
print("available actions: ", available_action)
# Chooses one of the available actions at random
def sample_next_action(available_actions_range):
  next_action = int(np.random.choice(available_action, 1))
  return next_action

action = sample_next_action(available_action)
print("next actions: ", action)

def update(current_state, action, gamma):
  max_index = np.where(Q[action, ] == np.max(Q[action, ]))[1]
  if max_index.shape[0] > 1:
    print("max_index: ",max_index)
    max_index = int(np.random.choice(max_index, size = 1))   #it will give output in aarray format
  else:
    print("max_index: ",max_index)
    max_index = int(max_index)   #It will be in the array format
  max_value = Q[action, max_index]
  Q[current_state, action] = R[current_state, action] + gamma * max_value
  if (np.max(Q) > 0):
    return(np.sum(Q / np.max(Q)*100))
  else:
    return (0)
# Updates the Q-Matrix according to the path chosen
print("Before Q matrix: ")
print(Q)
update(initial_state, action, gamma)
print("After Q matrix: ")
print(Q)
scores = []
for i in range(1000):
  current_state = np.random.randint(0, int(Q.shape[0]))
```

```
  available_action = available_actions(current_state)
  action = sample_next_action(available_action)
  score = update(current_state, action, gamma)
  scores.append(score)

print("Trained Q matrix:")
print(Q / np.max(Q)*100)
# You can uncomment the above two lines to view the trained Q matrix

pl.plot(scores)
pl.xlabel('No of iterations')
pl.ylabel('Reward gained')
pl.show()
current_state = 2
steps = [current_state]

while current_state != 10:

  next_step_index = np.where(Q[current_state, ] == np.max(Q[current_state, ]))[1]
  if next_step_index.shape[0] > 1:
    next_step_index = int(np.random.choice(next_step_index, size = 1))
  else:
    next_step_index = int(next_step_index)
  steps.append(next_step_index)
  current_state = next_step_index

print("Most efficient path:")
print(steps)
```

Results:




```
(0, 1)
(1, 5)
(5, 6)
(5, 4)
(1, 2)
(1, 3)
(9, 10)
(2, 4)
(0, 6)
(6, 7)
(8, 9)
(7, 8)
(1, 7)
(3, 9)
(10, 8)
(10, 10)
[[ -1.  0.  -1.  -1.  -1.  -1.  0.  -1.  -1.  -1.  -1.]
 [ 0.  -1.  0.  0.  -1.  0.  -1.  0.  -1.  -1.  -1.]
 [ -1.  0.  -1.  -1.  0.  -1.  -1.  -1.  -1.  -1.  -1.]
 [ -1.  0.  -1.  -1.  -1.  -1.  -1.  -1.  -1.  0.  -1.]
 [ -1.  -1.  0.  -1.  -1.  0.  -1.  -1.  -1.  -1.  -1.]
 [ -1.  0.  -1.  -1.  0.  -1.  0.  -1.  -1.  -1.  -1.]
 [ 0.  -1.  -1.  -1.  -1.  0.  -1.  0.  -1.  -1.  -1.]
```

```
[ -1.  0.  -1.  -1.  -1.  -1.  0.  -1.  0.  -1.  -1.]
```

```
 [ -1.  -1.  -1.  -1.  -1.  -1.  -1.   0.  -1.   0. 100.]
 [ -1.  -1.  -1.   0.  -1.  -1.  -1.  -1.   0.  -1. 100.]
 [ -1.  -1.  -1.  -1.  -1.  -1.  -1.  -1.   0.   0. 100.]]
available actions:  [0 2 3 5 7]
next actions:  7
Before Q matrix:
[[0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]]
max_index:  [ 0  1  2  3  4  5  6  7  8  9 10]
After Q matrix:
[[0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]]
max_index:  [ 0  1  2  3  4  5  6  7  8  9 10]
max_index:  [ 0  1  2  3  4  5  6  7  8  9 10]
max_index:  [ 0  1  2  3  4  5  6  7  8  9 10]
max_index:  [ 0  1  2  3  4  5  6  7  8  9 10]
max_index:  [ 0  1  2  3  4  5  6  7  8  9 10]
max_index:  [ 0  1  2  3  4  5  6  7  8  9 10]
max_index:  [ 0  1  2  3  4  5  6  7  8  9 10]
max_index:  [ 0  1  2  3  4  5  6  7  8  9 10]
max_index:  [ 0  1  2  3  4  5  6  7  8  9 10]
max_index:  [ 0  1  2  3  4  5  6  7  8  9 10]
max_index:  [10]
max_index:  [ 0  1  2  3  4  5  6  7  8  9 10]
max_index:  [10]
```

```
max_index:  [10]
max_index:  [ 0  1  2  3  4  5  6  7  8  9 10]
max_index:  [10]
max_index:  [ 0  1  2  3  4  5  6  7  8  9 10]
max_index:  [10]
max_index:  [ 0  1  2  3  4  5  6  7  8  9 10]
max_index:  [ 0  1  2  3  4  5  6  7  8  9 10]
max_index:  [ 0  1  2  3  4  5  6  7  8  9 10]
max_index:  [ 0  1  2  3  4  5  6  7  8  9 10]
max_index:  [ 0  1  2  3  4  5  6  7  8  9 10]
max_index:  [8 9]
max_index:  [ 0  1  2  3  4  5  6  7  8  9 10]
max_index:  [ 0  1  2  3  4  5  6  7  8  9 10]
max_index:  [ 0  1  2  3  4  5  6  7  8  9 10]
max_index:  [ 0  1  2  3  4  5  6  7  8  9 10]
max_index:  [10]
max_index:  [10]
max_index:  [ 0  1  2  3  4  5  6  7  8  9 10]
max_index:  [10]
max_index:  [ 0  1  2  3  4  5  6  7  8  9 10]
max_index:  [ 0  1  2  3  4  5  6  7  8  9 10]
max_index:  [10]
max_index:  [ 0  1  2  3  4  5  6  7  8  9 10]
max_index:  [9]
max_index:  [ 0  1  2  3  4  5  6  7  8  9 10]
max_index:  [ 0  1  2  3  4  5  6  7  8  9 10]
max_index:  [8]
max_index:  [ 0  1  2  3  4  5  6  7  8  9 10]
max_index:  [ 0  1  2  3  4  5  6  7  8  9 10]
max_index:  [ 0  1  2  3  4  5  6  7  8  9 10]
max_index:  [8]
max_index:  [ 0  1  2  3  4  5  6  7  8  9 10]
max_index:  [ 0  1  2  3  4  5  6  7  8  9 10]
max_index:  [ 0  1  2  3  4  5  6  7  8  9 10]
max_index:  [ 0  1  2  3  4  5  6  7  8  9 10]
max_index:  [10]
max_index:  [10]
max_index:  [ 0  1  2  3  4  5  6  7  8  9 10]
max_index:  [ 0  1  2  3  4  5  6  7  8  9 10]
max_index:  [ 0  1  2  3  4  5  6  7  8  9 10]
max_index:  [8]
max_index:  [ 0  1  2  3  4  5  6  7  8  9 10]
max_index:  [10]
```

```
max_index: [ 0  1  2  3  4  5  6  7  8  9 10]
max_index: [ 0  1  2  3  4  5  6  7  8  9 10]
max_index: [10]
max_index: [10]
max_index: [ 0  1  2  3  4  5  6  7  8  9 10]
max_index: [ 0  1  2  3  4  5  6  7  8  9 10]
max_index: [ 0  1  2  3  4  5  6  7  8  9 10]
max_index: [ 0  1  2  3  4  5  6  7  8  9 10]
max_index: [9]
max_index: [ 0  1  2  3  4  5  6  7  8  9 10]
max_index: [ 0  1  2  3  4  5  6  7  8  9 10]
max_index: [8]
max_index: [3]
max_index: [3]
max_index: [9]
max_index: [ 0  1  2  3  4  5  6  7  8  9 10]
max_index: [3]
max_index: [10]
max_index: [ 0  1  2  3  4  5  6  7  8  9 10]
max_index: [10]
max_index: [ 0  1  2  3  4  5  6  7  8  9 10]
max_index: [ 0  1  2  3  4  5  6  7  8  9 10]
max_index: [10]
max_index: [3]
max_index: [ 0  1  2  3  4  5  6  7  8  9 10]
max_index: [10]
max_index: [ 0  1  2  3  4  5  6  7  8  9 10]
max_index: [3]
max_index: [10]
max_index: [9]
max_index: [1]
max_index: [10]
max_index: [ 0  1  2  3  4  5  6  7  8  9 10]
max_index: [10]
max_index: [3]
max_index: [8]
max_index: [7]
max_index: [10]
max_index: [3]
max_index: [10]
max_index: [1]
max_index: [10]
max_index: [9]
```

```
max_index:  [7]
max_index:  [ 0  1  2  3  4  5  6  7  8  9 10]
max_index:  [10]
max_index:  [10]
max_index:  [3]
max_index:  [1]
max_index:  [10]
max_index:  [1]
max_index:  [9]
max_index:  [3]
max_index:  [6]
max_index:  [3]
max_index:  [7]
max_index:  [5]
max_index:  [10]
max_index:  [6]
max_index:  [10]
max_index:  [10]
max_index:  [1]
max_index:  [9]
max_index:  [10]
max_index:  [1]
max_index:  [3]
max_index:  [6]
max_index:  [10]
max_index:  [3]
max_index:  [10]
max_index:  [7]
max_index:  [6]
max_index:  [1]
max_index:  [3]
max_index:  [8]
max_index:  [6]
max_index:  [6]
max_index:  [5]
max_index:  [5]
max_index:  [3]
max_index:  [3]
max_index:  [5]
max_index:  [6]
max_index:  [8]
max_index:  [1]
max_index:  [5]
```

```
max_index:  [7]
max_index:  [3]
max_index:  [9]
max_index:  [3]
max_index:  [10]
max_index:  [3]
max_index:  [10]
max_index:  [10]
max_index:  [3]
max_index:  [10]
max_index:  [9]
max_index:  [10]
max_index:  [10]
max_index:  [10]
max_index:  [10]
max_index:  [5]
max_index:  [9]
max_index:  [10]
max_index:  [3]
max_index:  [8]
max_index:  [9]
max_index:  [9]
max_index:  [3]
max_index:  [1 6]
max_index:  [1]
max_index:  [3]
max_index:  [1]
max_index:  [7]
max_index:  [8]
max_index:  [5]
max_index:  [10]
max_index:  [1 6]
max_index:  [3]
max_index:  [8]
max_index:  [10]
max_index:  [10]
max_index:  [1]
max_index:  [8]
max_index:  [10]
max_index:  [3 7]
max_index:  [3 7]
max_index:  [10]
max_index:  [10]
```

```
max_index:  [10]
max_index:  [9]
max_index:  [6]
max_index:  [1 6]
max_index:  [1 6]
max_index:  [3]
max_index:  [10]
max_index:  [10]
max_index:  [3]
max_index:  [8]
max_index:  [10]
max_index:  [10]
max_index:  [10]
max_index:  [6]
max_index:  [5]
max_index:  [1]
max_index:  [6]
max_index:  [3]
max_index:  [5]
max_index:  [10]
max_index:  [6]
max_index:  [10]
max_index:  [10]
max_index:  [10]
max_index:  [9]
max_index:  [8]
max_index:  [3]
max_index:  [10]
max_index:  [10]
max_index:  [5]
max_index:  [3]
max_index:  [7]
max_index:  [10]
max_index:  [7]
max_index:  [9]
max_index:  [3]
max_index:  [10]
max_index:  [7]
max_index:  [6]
max_index:  [1 6]
max_index:  [9]
max_index:  [5]
max_index:  [10]
```

```
max_index: [10]
max_index: [10]
max_index: [3]
max_index: [10]
max_index: [3]
max_index: [1]
max_index: [3]
max_index: [7]
max_index: [5]
max_index: [1]
max_index: [10]
max_index: [10]
max_index: [3]
max_index: [7]
max_index: [8]
max_index: [1]
max_index: [10]
max_index: [10]
max_index: [1]
max_index: [1]
max_index: [10]
max_index: [8]
max_index: [7]
max_index: [7]
max_index: [10]
max_index: [1]
max_index: [1]
max_index: [7]
max_index: [5]
max_index: [5]
max_index: [3]
max_index: [7]
max_index: [3]
max_index: [5]
max_index: [10]
max_index: [3]
max_index: [8]
max_index: [5]
max_index: [3]
max_index: [3]
max_index: [1]
max_index: [7]
max_index: [3]
```

max_index: [3]
max_index: [8]
max_index: [9]
max_index: [3]
max_index: [10]
max_index: [10]
max_index: [3]
max_index: [1]
max_index: [10]
max_index: [7]
max_index: [3]
max_index: [6]
max_index: [10]
max_index: [3]
max_index: [3]
max_index: [1]
max_index: [10]
max_index: [6]
max_index: [8]
max_index: [10]
max_index: [7]
max_index: [3]
max_index: [8]
max_index: [6]
max_index: [10]
max_index: [3]
max_index: [1]
max_index: [6]
max_index: [10]
max_index: [10]
max_index: [10]
max_index: [7]
max_index: [10]
max_index: [3]
max_index: [7]
max_index: [6]
max_index: [7]
max_index: [6]
max_index: [6]
max_index: [8]
max_index: [6]
max_index: [10]
max_index: [6]

```
max_index:  [1]
max_index:  [3]
max_index:  [10]
max_index:  [7]
max_index:  [3]
max_index:  [9]
max_index:  [10]
max_index:  [1]
max_index:  [6]
max_index:  [9]
max_index:  [10]
max_index:  [8]
max_index:  [1]
max_index:  [6]
max_index:  [8]
max_index:  [5]
max_index:  [10]
max_index:  [10]
max_index:  [10]
max_index:  [3]
max_index:  [7]
max_index:  [7]
max_index:  [8]
max_index:  [7]
max_index:  [10]
max_index:  [5]
max_index:  [6]
max_index:  [3]
max_index:  [3]
max_index:  [3]
max_index:  [3]
max_index:  [10]
max_index:  [5]
max_index:  [3]
max_index:  [10]
max_index:  [6]
max_index:  [3]
max_index:  [10]
max_index:  [5]
max_index:  [10]
max_index:  [3]
max_index:  [3]
max_index:  [10]
```

```
max_index:  [6]
max_index:  [6]
max_index:  [3]
max_index:  [7]
max_index:  [6]
max_index:  [10]
max_index:  [10]
max_index:  [3]
max_index:  [3]
max_index:  [1]
max_index:  [1]
max_index:  [7]
max_index:  [10]
max_index:  [10]
max_index:  [6]
max_index:  [10]
max_index:  [3]
max_index:  [10]
max_index:  [10]
max_index:  [9]
max_index:  [7]
max_index:  [6]
max_index:  [10]
max_index:  [9]
max_index:  [3]
max_index:  [8]
max_index:  [10]
max_index:  [3]
max_index:  [3]
max_index:  [1 6]
max_index:  [9]
max_index:  [1 6]
max_index:  [10]
max_index:  [8]
max_index:  [9]
max_index:  [7]
max_index:  [3]
max_index:  [3]
max_index:  [7]
max_index:  [1 6]
max_index:  [7]
max_index:  [1 6]
max_index:  [10]
```

```
max_index:  [3]
max_index:  [10]
max_index:  [3]
max_index:  [7]
max_index:  [10]
max_index:  [8]
max_index:  [10]
max_index:  [1]
max_index:  [10]
max_index:  [3]
max_index:  [3]
max_index:  [3]
max_index:  [3]
max_index:  [9]
max_index:  [1 6]
max_index:  [9]
max_index:  [3]
max_index:  [10]
max_index:  [10]
max_index:  [3]
max_index:  [10]
max_index:  [9]
max_index:  [1 6]
max_index:  [5]
max_index:  [5]
max_index:  [3]
max_index:  [5]
max_index:  [3]
max_index:  [7]
max_index:  [7]
max_index:  [3]
max_index:  [3]
max_index:  [10]
max_index:  [3]
max_index:  [8]
max_index:  [9]
max_index:  [1 6]
max_index:  [1 6]
max_index:  [10]
max_index:  [10]
max_index:  [7]
max_index:  [7]
max_index:  [3]
```

```
max_index:  [3]
max_index:  [9]
max_index:  [10]
max_index:  [6]
max_index:  [3]
max_index:  [5]
max_index:  [3]
max_index:  [5]
max_index:  [10]
max_index:  [9]
max_index:  [1]
max_index:  [2 5]
max_index:  [9]
max_index:  [1]
max_index:  [7]
max_index:  [2 5]
max_index:  [3]
max_index:  [2 5]
max_index:  [3]
max_index:  [8]
max_index:  [3]
max_index:  [8]
max_index:  [8]
max_index:  [10]
max_index:  [3]
max_index:  [10]
max_index:  [10]
max_index:  [10]
max_index:  [3]
max_index:  [1 6]
max_index:  [10]
max_index:  [7]
max_index:  [10]
max_index:  [3]
max_index:  [1]
max_index:  [10]
max_index:  [9]
max_index:  [8]
max_index:  [1 6]
max_index:  [2 5]
max_index:  [9]
max_index:  [3 7]
max_index:  [10]
```

```
max_index:  [9]
max_index:  [10]
max_index:  [3 7]
max_index:  [8]
max_index:  [1 6]
max_index:  [6]
max_index:  [3 7]
max_index:  [7]
max_index:  [1 6]
max_index:  [1 6]
max_index:  [1 6]
max_index:  [10]
max_index:  [10]
max_index:  [3 7]
max_index:  [1 6]
max_index:  [10]
max_index:  [10]
max_index:  [10]
max_index:  [7]
max_index:  [7]
max_index:  [8]
max_index:  [10]
max_index:  [2 5]
max_index:  [1 6]
max_index:  [10]
max_index:  [9]
max_index:  [9]
max_index:  [10]
max_index:  [7]
max_index:  [1]
max_index:  [7]
max_index:  [3]
max_index:  [1 6]
max_index:  [10]
max_index:  [2 5]
max_index:  [9]
max_index:  [7]
max_index:  [3]
max_index:  [7]
max_index:  [10]
max_index:  [2 5]
max_index:  [3]
max_index:  [6]
```

```
max_index:  [3]
max_index:  [10]
max_index:  [6]
max_index:  [3]
max_index:  [10]
max_index:  [9]
max_index:  [3]
max_index:  [7]
max_index:  [3]
max_index:  [3]
max_index:  [1]
max_index:  [10]
max_index:  [10]
max_index:  [9]
max_index:  [10]
max_index:  [7]
max_index:  [3]
max_index:  [1 6]
max_index:  [3]
max_index:  [8]
max_index:  [10]
max_index:  [1 6]
max_index:  [10]
max_index:  [10]
max_index:  [10]
max_index:  [3]
max_index:  [8]
max_index:  [5]
max_index:  [7]
max_index:  [3]
max_index:  [10]
max_index:  [1]
max_index:  [1]
max_index:  [2 5]
max_index:  [10]
max_index:  [10]
max_index:  [1 6]
max_index:  [3]
max_index:  [9]
max_index:  [8]
max_index:  [6]
max_index:  [10]
max_index:  [9]
```

```
max_index:  [8]
max_index:  [3]
max_index:  [10]
max_index:  [3]
max_index:  [10]
max_index:  [8]
max_index:  [10]
max_index:  [3]
max_index:  [7]
max_index:  [10]
max_index:  [8]
max_index:  [1]
max_index:  [7]
max_index:  [3]
max_index:  [7]
max_index:  [1]
max_index:  [7]
max_index:  [10]
max_index:  [7]
max_index:  [7]
max_index:  [10]
max_index:  [7]
max_index:  [7]
max_index:  [3]
max_index:  [2 5]
max_index:  [2 5]
max_index:  [1]
max_index:  [1]
max_index:  [9]
max_index:  [8]
max_index:  [2 5]
max_index:  [10]
max_index:  [3]
max_index:  [3]
max_index:  [3]
max_index:  [8]
max_index:  [9]
max_index:  [1]
max_index:  [3]
max_index:  [10]
max_index:  [1]
max_index:  [3]
max_index:  [1]
```

```
max_index:  [10]
max_index:  [10]
max_index:  [10]
max_index:  [8]
max_index:  [1]
max_index:  [10]
max_index:  [10]
max_index:  [10]
max_index:  [2 5]
max_index:  [9]
max_index:  [10]
max_index:  [10]
max_index:  [1]
max_index:  [10]
max_index:  [7]
max_index:  [1]
max_index:  [2 5]
max_index:  [10]
max_index:  [3]
max_index:  [8]
max_index:  [10]
max_index:  [2 5]
max_index:  [7]
max_index:  [3]
max_index:  [10]
max_index:  [3]
max_index:  [8]
max_index:  [10]
max_index:  [10]
max_index:  [9]
max_index:  [10]
max_index:  [7]
max_index:  [10]
max_index:  [3]
max_index:  [10]
max_index:  [7]
max_index:  [2 5]
max_index:  [10]
max_index:  [10]
max_index:  [10]
max_index:  [10]
max_index:  [10]
max_index:  [3]
```

max_index: [10]
max_index: [1]
max_index: [10]
max_index: [7]
max_index: [1]
max_index: [2 5]
max_index: [1]
max_index: [10]
max_index: [10]
max_index: [9]
max_index: [3]
max_index: [10]
max_index: [1]
max_index: [8]
max_index: [9]
max_index: [10]
max_index: [3]
max_index: [10]
max_index: [3]
max_index: [10]
max_index: [3]
max_index: [10]
max_index: [1]
max_index: [10]
max_index: [3]
max_index: [8]
max_index: [7]
max_index: [3]
max_index: [1]
max_index: [8]
max_index: [1]
max_index: [8]
max_index: [9]
max_index: [10]
max_index: [10]
max_index: [7]
max_index: [7]
max_index: [6]
max_index: [1]
max_index: [7]
max_index: [7]
max_index: [7]
max_index: [7]

```
max_index:  [1]
max_index:  [1]
max_index:  [7]
max_index:  [7]
max_index:  [7]
max_index:  [1]
max_index:  [2 5]
max_index:  [1]
max_index:  [6]
max_index:  [6]
max_index:  [7]
max_index:  [10]
max_index:  [10]
max_index:  [10]
max_index:  [1]
max_index:  [7]
max_index:  [7]
max_index:  [7]
max_index:  [8]
max_index:  [7]
max_index:  [1 6]
max_index:  [10]
max_index:  [2 5]
max_index:  [8]
max_index:  [10]
max_index:  [7]
max_index:  [10]
max_index:  [7]
max_index:  [7]
max_index:  [7]
max_index:  [7]
max_index:  [10]
max_index:  [10]
max_index:  [2 5]
max_index:  [1 6]
max_index:  [1 6]
max_index:  [9]
max_index:  [1 6]
max_index:  [8]
max_index:  [8]
max_index:  [1 6]
max_index:  [10]
max_index:  [3 7]
```

```
max_index:  [10]
max_index:  [8]
max_index:  [10]
max_index:  [10]
max_index:  [7]
max_index:  [7]
max_index:  [1]
max_index:  [2 5]
max_index:  [3 7]
max_index:  [10]
max_index:  [3 7]
max_index:  [10]
max_index:  [10]
max_index:  [10]
max_index:  [3 7]
max_index:  [10]
max_index:  [8]
max_index:  [7]
max_index:  [1]
max_index:  [8]
max_index:  [8]
max_index:  [10]
max_index:  [9]
max_index:  [3 7]
max_index:  [7]
max_index:  [1]
max_index:  [8]
max_index:  [3 7]
max_index:  [2 5]
max_index:  [10]
max_index:  [10]
max_index:  [3 7]
max_index:  [7]
max_index:  [1 6]
max_index:  [2 5]
max_index:  [1]
max_index:  [10]
max_index:  [1 6]
max_index:  [10]
max_index:  [7]
max_index:  [3 7]
max_index:  [2 5]
max_index:  [10]
```

```
max_index:  [10]
max_index:  [8]
max_index:  [3 7]
max_index:  [1 6]
max_index:  [3 7]
max_index:  [3 7]
max_index:  [1 6]
max_index:  [7]
max_index:  [10]
max_index:  [10]
max_index:  [8]
max_index:  [2 5]
max_index:  [3 7]
max_index:  [10]
max_index:  [1 6]
max_index:  [10]
max_index:  [7]
max_index:  [7]
max_index:  [3 7]
max_index:  [10]
max_index:  [10]
max_index:  [3 7]
max_index:  [7]
max_index:  [10]
max_index:  [1 6]
max_index:  [10]
max_index:  [10]
max_index:  [10]
max_index:  [1 6]
max_index:  [7]
max_index:  [10]
max_index:  [1 6]
max_index:  [10]
max_index:  [1]
max_index:  [8]
max_index:  [1]
max_index:  [2 5]
max_index:  [1 6]
max_index:  [10]
max_index:  [10]
max_index:  [1 6]
max_index:  [3 7]
max_index:  [1 6]
```

```
max_index:  [10]
max_index:  [8]
max_index:  [3 7]
max_index:  [10]
max_index:  [10]
max_index:  [3 7]
max_index:  [3 7]
max_index:  [10]
max_index:  [3 7]
max_index:  [10]
max_index:  [10]
max_index:  [10]
max_index:  [3 7]
max_index:  [3 7]
max_index:  [2 5]
max_index:  [9]
max_index:  [2 5]
max_index:  [3 7]
max_index:  [10]
max_index:  [9]
max_index:  [10]
max_index:  [7]
max_index:  [3 7]
max_index:  [1 6]
max_index:  [10]
max_index:  [3 7]
max_index:  [1 6]
max_index:  [1]
max_index:  [1 6]
max_index:  [10]
max_index:  [3 7]
max_index:  [9]
max_index:  [10]
max_index:  [3 7]
max_index:  [2 5]
max_index:  [3 7]
max_index:  [3 7]
max_index:  [1 6]
max_index:  [2 5]
max_index:  [3 7]
max_index:  [1 6]
max_index:  [8]
max_index:  [10]
```

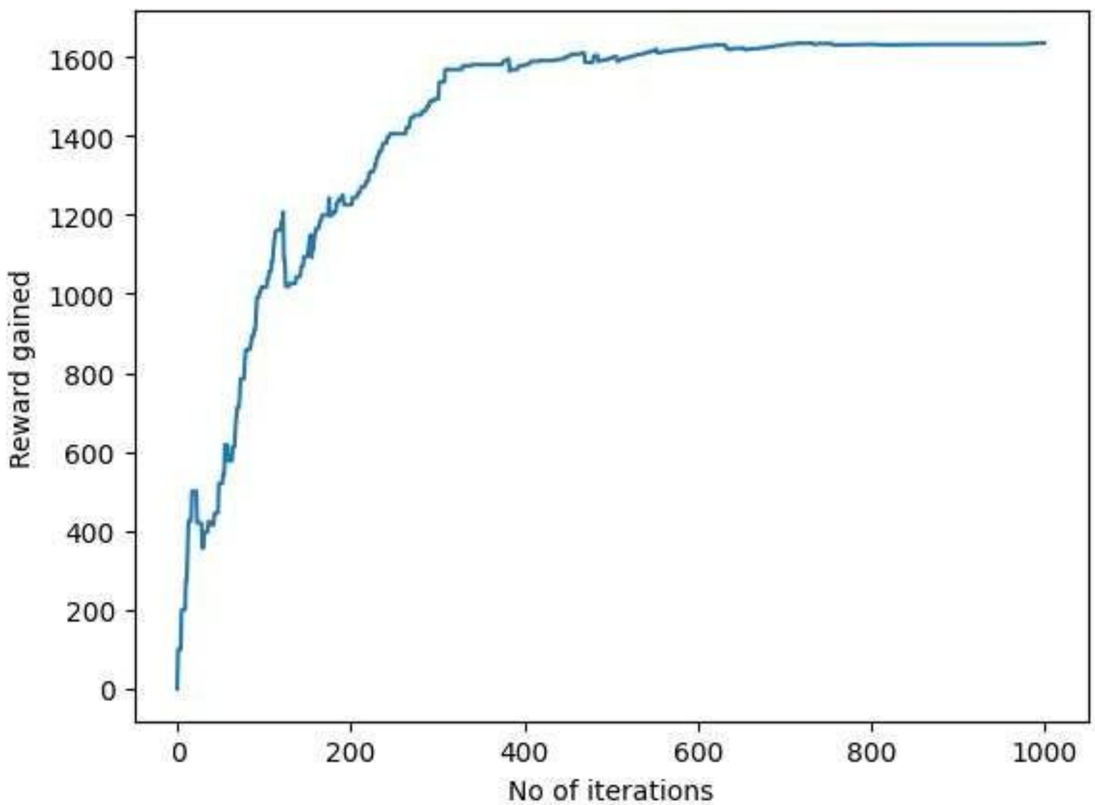
```
max_index:  [8]
max_index:  [3 7]
max_index:  [1 6]
max_index:  [1 6]
max_index:  [1 6]
max_index:  [1 6]
max_index:  [2 5]
max_index:  [1]
max_index:  [3 7]
max_index:  [1]
max_index:  [3 7]
max_index:  [3 7]
max_index:  [8]
max_index:  [7]
max_index:  [3 7]
max_index:  [3 7]
max_index:  [2 5]
max_index:  [10]
max_index:  [3 7]
max_index:  [1 6]
max_index:  [10]
max_index:  [7]
max_index:  [10]
max_index:  [3 7]
max_index:  [1 6]
max_index:  [3 7]
max_index:  [3 7]
max_index:  [3 7]
max_index:  [7]
max_index:  [10]
max_index:  [10]
max_index:  [3 7]
max_index:  [2 5]
max_index:  [10]
max_index:  [3 7]
max_index:  [1]
max_index:  [9]
max_index:  [3 7]
max_index:  [1 6]
max_index:  [2 5]
max_index:  [1 6]
max_index:  [2 5]
max_index:  [1]
```

```
max_index:  [7]
max_index:  [3 7]
max_index:  [7]
max_index:  [3 7]
max_index:  [3 7]
max_index:  [3 7]
max_index:  [10]
max_index:  [2 5]
max_index:  [3 7]
max_index:  [3 7]
max_index:  [3 7]
max_index:  [10]
max_index:  [10]
max_index:  [10]
max_index:  [7]
max_index:  [1]
max_index:  [10]
max_index:  [2 5]
max_index:  [3 7]
max_index:  [7]
max_index:  [1 6]
max_index:  [1 6]
max_index:  [1 6]
max_index:  [1]
max_index:  [1 6]
max_index:  [1 6]
max_index:  [3 7]
max_index:  [3 7]
max_index:  [8]
max_index:  [10]
max_index:  [1 6]
max_index:  [10]
max_index:  [1 6]
max_index:  [1 6]
max_index:  [3 7]
max_index:  [3 7]
max_index:  [1 6]
max_index:  [10]
max_index:  [10]
max_index:  [3 7]
max_index:  [2 5]
max_index:  [10]
max_index:  [3 7]
```

```
max_index:  [2 5]
max_index:  [2 5]
max_index:  [1 6]
max_index:  [10]
max_index:  [1]
max_index:  [1]
max_index:  [1 6]
max_index:  [1 6]
max_index:  [9]
max_index:  [7]
max_index:  [3 7]
max_index:  [7]
max_index:  [10]
max_index:  [10]
max_index:  [2 5]
max_index:  [7]
max_index:  [7]
max_index:  [3 7]
max_index:  [1]
max_index:  [10]
max_index:  [8]
max_index:  [8]
max_index:  [7]
max_index:  [9]
max_index:  [6]
max_index:  [7]
max_index:  [7]
max_index:  [7]
max_index:  [8]
max_index:  [7]
max_index:  [10]
max_index:  [7]
max_index:  [2 5]
max_index:  [10]
max_index:  [6]
max_index:  [10]
max_index:  [8]
max_index:  [7]
max_index:  [1]
max_index:  [8]
max_index:  [7]
Trained Q matrix:
[[  0.         42.18122409   0.          0.          0.
```

```
  0.         42.18122409   0.          0.          0.
  0.       ]
 [ 31.40399258   0.         31.63591807  55.82932014   0.
  31.40399258   0.         56.24163212   0.          0.
  0.       ]
 [  0.         42.18122409   0.          0.         23.55299443
  0.          0.          0.          0.          0.
  0.       ]
 [  0.         42.18122409   0.          0.          0.
  0.          0.          0.          0.         75.
  0.       ]
 [  0.          0.         31.40399258   0.          0.
  31.63591807   0.          0.          0.          0.
  0.       ]
 [  0.         41.8719901    0.          0.         23.55299443
  0.         42.18122409   0.          0.          0.
  0.       ]
 [ 31.40399258   0.          0.          0.          0.
  31.63591807   0.         56.24163212   0.          0.
  0.       ]
 [  0.         42.18122409   0.          0.          0.
  0.         42.18122409   0.         74.98884283   0.
  0.       ]
 [  0.          0.          0.          0.          0.
  0.          0.         56.24163212   0.         74.98884283
 100.        ]
 [  0.          0.          0.         56.24163212   0.
  0.          0.          0.         74.98884283   0.
 100.        ]
 [  0.          0.          0.          0.          0.
  0.          0.          0.         75.         74.927685
  99.98512377]]
/tmp/ipykernel_23441/3591224371.py:52: DeprecationWarning: Conversion of an array with ndim > 0 to a scalar is deprecated, and will error in future. Ensure you extract a single element from your array before performing this operation. (Deprecated NumPy 1.25.)
  next_action = int(np.random.choice(available_action, 1))
/tmp/ipykernel_23441/3591224371.py:63: DeprecationWarning: Conversion of an array with ndim > 0 to a scalar is deprecated, and will error in future. Ensure you extract a single element from your array before performing this operation. (Deprecated NumPy 1.25.)
  max_index = int(np.random.choice(max_index, size = 1))   #it will give output in aarray format
/tmp/ipykernel_23441/3591224371.py:66: DeprecationWarning: Conversion of an array with ndim > 0 to a scalar is deprecated, and will error in future. Ensure you extract a single element from your array before performing this operation. (Deprecated NumPy 1.25.)
```

max_index = int(max_index)   #It will be in the array format



Most efficient path:
[2, 1, 7, 8, 10]
/tmp/ipykernel_23441/3591224371.py:104: DeprecationWarning: Conversion of an array with ndim > 0 to a scalar is deprecated, and will error in future. Ensure you extract a single element from your array before performing this operation. (Deprecated NumPy 1.25.)
  next_step_index = int(next_step_index)

Observation:

The program effectively solves a Sudoku puzzle using the backtracking technique. It fills empty cells (denoted by 0) with digits from 1 to 9 while maintaining all Sudoku rules. The isSafe() function verifies whether a number can be placed in a specific row, column, and 3×3 sub-grid without breaking any constraints. If the number is valid, it is temporarily placed in the cell, and the algorithm recursively continues to solve the remaining cells.

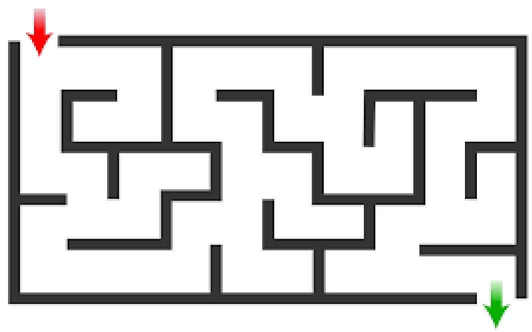
During execution, the algorithm tries different possible values for each empty cell. If a selected value causes a conflict at a later stage, the algorithm backtracks by removing that value and testing the next option. This process is repeated until all the cells are successfully filled with valid numbers.

The observation indicates that the algorithm successfully finds a valid solution by systematically exploring all possible combinations. It ensures correctness by placing only those numbers that satisfy Sudoku constraints and revisiting earlier decisions whenever a conflict occurs. However, due to its recursive and exhaustive search nature, the algorithm may require more time to solve highly complex puzzles.

Finally, the output presents a fully completed and valid Sudoku grid, confirming that the algorithm has successfully solved the puzzle.

Post Lab Exercise:

Implement the reinforcement learning to train an agent to travel through the maze given in the image below. Also, explain the step-by-step procedure to train the agent and get the output sequence path for the same.



Step-wise Procedure to Train an Agent for Maze using Reinforcement Learning

Step 1: Define the Problem Environment

The given maze is considered as the environment in which the agent operates. Each position (cell) in the maze represents a state. The agent starts from the initial position (start point) and aims to reach the goal position (end point). The agent can move in four possible directions: up, down, left, and right, while avoiding walls.

Step 2: Define States and Actions

Each cell in the maze is treated as a state. The possible actions available to the agent in each state are movement directions such as up, down, left, and right. If a movement leads to a wall or outside the maze, it is considered an invalid action.

Step 3: Define Reward Function

A reward system is defined to guide the agent's learning. A positive reward (e.g., +10 or +100) is given when the agent reaches the goal. A small negative reward (e.g., −1) is given for each step to encourage shorter paths. A larger negative reward (e.g., −10) is assigned when the agent hits a wall or makes an invalid move.

Step 4: Initialize Q-Table

A Q-table is created where rows represent states and columns represent actions. Initially, all Q-values are set to zero. This table will be updated during training to store the usefulness of each action in each state.

Step 5: Set Learning Parameters

Important parameters are defined, such as learning rate (α), discount factor (γ), and exploration rate (ε). The learning rate determines how much new information overrides old information, the discount factor determines the importance of future rewards, and the exploration rate controls the balance between exploration and exploitation.

Step 6: Training the Agent (Learning Process)

The agent is trained over multiple episodes. In each episode, the agent starts from the initial state and selects actions using an ε-greedy policy, where it either explores randomly or exploits the best-known action. After taking an action, the agent moves to a new state and receives a reward. The Q-value is updated using the Q-learning formula. This process continues until the agent reaches the goal. Repeating this over many episodes allows the agent to learn the optimal path.

Step 7: Policy Formation

After sufficient training, the Q-values converge, and the agent forms an optimal policy. This policy determines the best action to take in each state to maximize the total reward.

Step 8: Extract the Optimal Path

Using the learned Q-table, the agent starts from the initial state and always selects the action with the highest Q- value. By following these actions step-by-step, the agent reaches the goal through the optimal path.

Step 9: Output Path Sequence

The final output is the sequence of steps taken by the agent from start to goal. This sequence represents the shortest or most efficient path learned by the agent through reinforcement learning